Table of Contents

Quick Start ---------- 2

Quick Start

The example code provided in this quick start guide is for educational and demonstration purposes only. It may not represent best practices for production use. This quick start was last updated for **UAI.LiteRTLM v2.1.0-preview.2**.

> ⊗ **IMPORTANT**
>
> **UAI.LiteRTLM v2.0.0+** is a complete rewrite of the package.
>
> - The API is **not compatible** with v1.x.
> - If you're upgrading from v1.x, read this Quick Start and the reference documentation before migrating.

Version Compatibility

LiteRT-LM is under active development, and its C API changes frequently. As a result, **each version of UAI.LiteRTLM is built against a specific LiteRT-LM release**.

The table below shows which LiteRT-LM version is used by each UAI.LiteRTLM release.

UAI.LiteRTLM	LiteRT-LM	Supported Platforms
2.1.0-preview.x	v0.15.0-alpha0 (`ad53ed1`)	Android (arm64)
2.0.0-preview.1	v0.14.0 (`80f301f`)	Android (arm64)

Since UAI.LiteRTLM uses the LiteRT-LM C API, it can theoretically support any platform that LiteRT-LM supports. However, prebuilt native binaries are currently only provided for **Android (arm64)**.

Contributions to add support for additional platforms are welcome. The native build script can be found at:

```
Native/build.sh
```

API Stability

Because LiteRT-LM's C API is still evolving, the UAI.LiteRTLM API may also change between releases.

Check the release notes before updating to a newer version.

GPU Inference

To support GPU inference on Android, add the following to the `<application>` group in your `AndroidManifest.xml` to enable OpenCL:

```
<uses-native-library android:name="libvndksupport.so" android:required="false"/>
<uses-native-library android:name="libOpenCL.so" android:required="false"/>
```

Example

The documentation is still a work in progress. The following example demonstrates how to:

```
using System.IO;
using UnityEngine;
using Uralstech.UAI.LiteRTLM;

private void RunConversation()
{
    string modelPath = Path.Join(Application.persistentDataPath, "model.litertlm");
    string cacheDir = Path.Join(Application.temporaryCachePath, "modelCache");
    Directory.CreateDirectory(cacheDir);

    using EngineSettings engineSettings = new(modelPath, BackendNames.GPU);
    engineSettings.SetCacheDir(cacheDir);
    engineSettings.EnableBenchmark();

    using Engine engine = new(engineSettings);
    using Conversation conversation = new(engine);

    Debug.Log("Engine and conversation created.");

    const string message = "{\"role\":\"user\",\"content\":\"What is the tallest building in
the world?\"}";
    using JsonResponse? result = conversation.SendMessage(message);

    Debug.Log("Model response: " + result?.GetString());

    using BenchmarkInfo? benchmarkInfo = conversation.GetBenchmarkInfo();
    if (benchmarkInfo == null) return;

    BenchmarkInfo.Turn[] prefillTurns = benchmarkInfo.GetPrefillTurns();
    BenchmarkInfo.Turn[] decodeTurns = benchmarkInfo.GetDecodeTurns();

    Debug.Log("Benchmark info:"
              + $"\n\tInitialization time: {benchmarkInfo.GetTotalInitTime()}"
```

```
            + $"\n\tTime to first token: {benchmarkInfo.GetTimeToFirstToken()}"
            + $"\n\tPrefill tokens per second: {prefillTurns[0].TokensPerSecond}, for
total: {prefillTurns[0].TokenCount} tokens"
            + $"\n\tDecode tokens per second: {decodeTurns[0].TokensPerSecond}, for total:
{decodeTurns[0].TokenCount} tokens"
    );
}
```